Flores Diaz Diego Adair 323294506

```
diego@LAPTOP-DR7RBISN MINGW64 ~/mpoo/mpoo-2027-1/entrega
$ java -cp out mazmorra.Main
Al empezar:
Conan (Caballero) 120/120
Merlin (Mago) 80/80
Legolas (Arquero) 100/100
Despues de una ronda:
Conan (Caballero) 110/120
Merlin (Mago) 62/80
Legolas (Arquero) 75/100
Espada: durabilidad 99
```

```
$ javac -cp out -d out src/trampas/TrampasDeAcceso.java
src\trampas\TrampasDeAcceso.java:10: error: vida has private access in Personaje
        heroe.vida = 9999;
              ^
src\trampas\TrampasDeAcceso.java:11: error: inventario has private access in Personaje
        heroe.inventario = new Inventario();
              ^
src\trampas\TrampasDeAcceso.java:14: error: durabilidad has private access in Herramienta
        espada.durabilidad = 1000000;
               ^
src\trampas\TrampasDeAcceso.java:17: error: herramientas has private access in Inventario
        bolsa.herramientas[0] = espada;
              ^
src\trampas\TrampasDeAcceso.java:18: error: cuantas has private access in Inventario
        bolsa.cuantas = 50;
              ^
5 errors
```

```
$ java -cp out trampas.Tramposo
BLOQUEADA  1) golpe negativo para curarse
BLOQUEADA  2) curarse arriba del tope de vida
BLOQUEADA  3) forjar una espada de 5000 de poder
BLOQUEADA  4) cargar una cuarta herramienta
BLOQUEADA  5) pedir la herramienta 99
BLOQUEADA  6) atacar sin ninguna herramienta
BLOQUEADA  7) revivir a un personaje muerto
```

```
.
+-- JUnit Platform Suite [OK]
+-- JUnit Jupiter [OK]
| '-- PersonajeTest [OK]
|   +-- caballeroConEspadaQuitaVeinticincoAArquero() [OK]
|   +-- magoCuraACompaneroHeridoAlAyudarlo() [OK]
|   +-- hachaCompartidaMantieneLaMismaInstanciaEnAmbosPersonajes() [OK]
|   +-- laVidaNuncaPasaDeLaMaxima() [OK]
|   +-- recibirDanioRestaLosPuntos() [X] expected: <70> but was: <50>
|   '-- cadaPersonajeTieneSuPropioInventarioIndependiente() [OK]
'-- JUnit Vintage [OK]

Failures (1):
  JUnit Jupiter:PersonajeTest:recibirDanioRestaLosPuntos()
    MethodSource [className = 'mazmorra.PersonajeTest', methodName = 'recibirDanioRestaLosPuntos', methodParameterTypes = '']
    => org.opentest4j.AssertionFailedError: expected: <70> but was: <50>
       org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
       org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow(AssertionFailureBuilder.java:132)
       org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
       org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:150)
       org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:145)
       org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:531)
       mazmorra.PersonajeTest.recibirDanioRestaLosPuntos(PersonajeTest.java:20)
       java.base/java.lang.reflect.Method.invoke(Method.java:580)
       java.base/java.util.ArrayList.forEach(ArrayList.java:1596)
       java.base/java.util.ArrayList.forEach(ArrayList.java:1596)

Test run finished after 66 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         6 tests found           ]
[         0 tests skipped         ]
[         6 tests started         ]
[         0 tests aborted         ]
[         5 tests successful      ]
[         1 tests failed          ]
```

La prueba en rojo es de la prueba 4b

Flores Diaz Diego Adair 323294506

1. Las cuatro relaciones: para herencia, composición, agregación y asociación, copia la línea de tu código donde se ve cada una y di qué le pasa a la parte si el dueño desaparece (para la herencia, qué hace super(...)).

1.Herencia:

```
Caballero extends Personaje {

   public Caballero(String nombre) {
       super(nombre, 120);
```

Si la clase padre desaparece, la clase hija no compilara porque hereda cosas de su padre, y super lo que hace es invocar al constructor del padre para crear ahora un hijo caballero.

2.Asociación:

```
public void atacar(Personaje objetivo) {
```

En este caso no pasaría nada, el objetivo es independiente y si el personaje que iba a atacar muere o desaparece el objetivo sigue existiendo sin ningún problema.

3.Agregación:

```
equipar(Herramienta h) {
   return this.inventario.agregar(h);
}
```

4.Composición:

```
this.inventario = new Inventario();
```

La parte desaparece junto con el dueño, porque el inventario se crea dentro del personaje al nacer, si el personaje muere, su inventario muere junto con el.

Tampoco pasaría nada, si el personaje muere o desaparece que es el dueño, la herramienta seguiría existiendo pero sin que la equipe el personaje.

2. Los huecos: ¿alguna trampa funcionó la primera vez? ¿Qué cambiaste? Si ninguna funcionó, elige una y explica qué línea de tu código la bloquea. ¿Por qué el diagrama no tiene setVida ni getHerramientas ?

```
public void recibirDanio(int puntos) {
   if (puntos > 0 && estaVivo()) {
       this.vida = limitar(this.vida - puntos, 0, this.vidaMaxima);
   }
```

Ninguna trampa funcionó y elegí la de la primera trampa y no sirvo porque en el código esta especificado que los puntos de daño que reciba debe ser mayor a 0 y también que el personaje tiene que estar vivo.

No hay servida para que nadie pueda cambiar la vida más que los métodos específicos para eso, ni getherramientas para que no puedas agregar nada sin antes pasar por el método que evita meter mas de las permitidas.

3. Una prueba roja: copia el mensaje del paso 4b y explica qué significa cada número y
quién estaba mal.

```
|   +-- laVidaNuncaPasaDeLaMaxima() [OK]
|   +-- recibirDanioRestaLosPuntos() [X] expected: <70> but was: <50>
```

El 70 significa que esperaba que esa fuera la vida restante del mago, pero el verdadero valor que tenía que salir era el número, lo que estaba mal fue calcular los puntos si en vez de poner 70 se hubieran puesto los 50 la prueba hubiera salido OK.